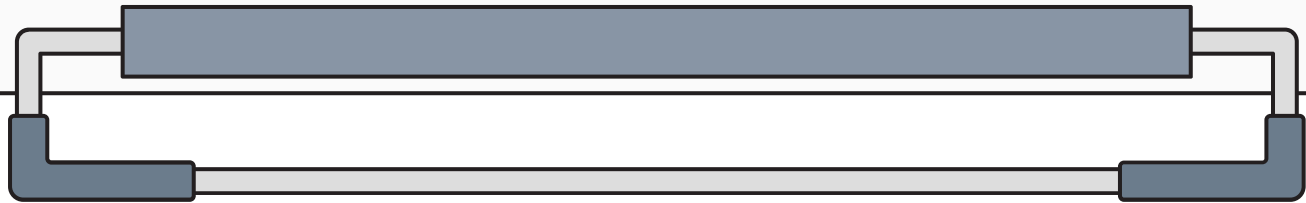


CRUD Application

2026-07-01 양지웅



목차

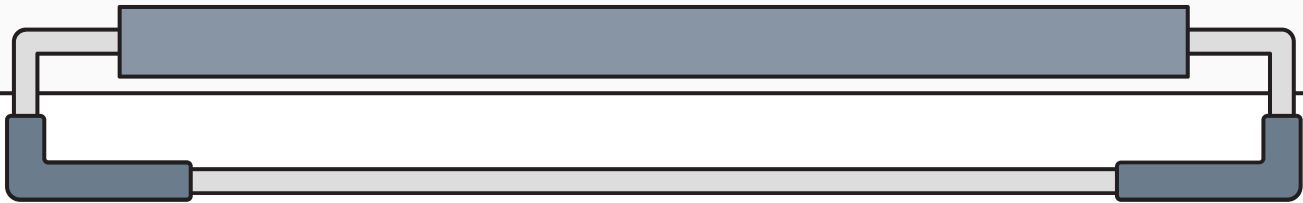
01 프로젝트 개요
개요 및 개발환경, 기술스택 소개

02 프로젝트 구조 및 DB 설계
프로젝트 구조 및 User1 테이블 설계

03 CRUD 기능 구현
데이터 CRUD 기능 소개

04 실행 결과
실행 결과에 대한 실제 화면 출력

05 결론
결론 및 개선사항



01 프로젝트 개요

CRUD Application 개발

User1 목록

[메인](#)

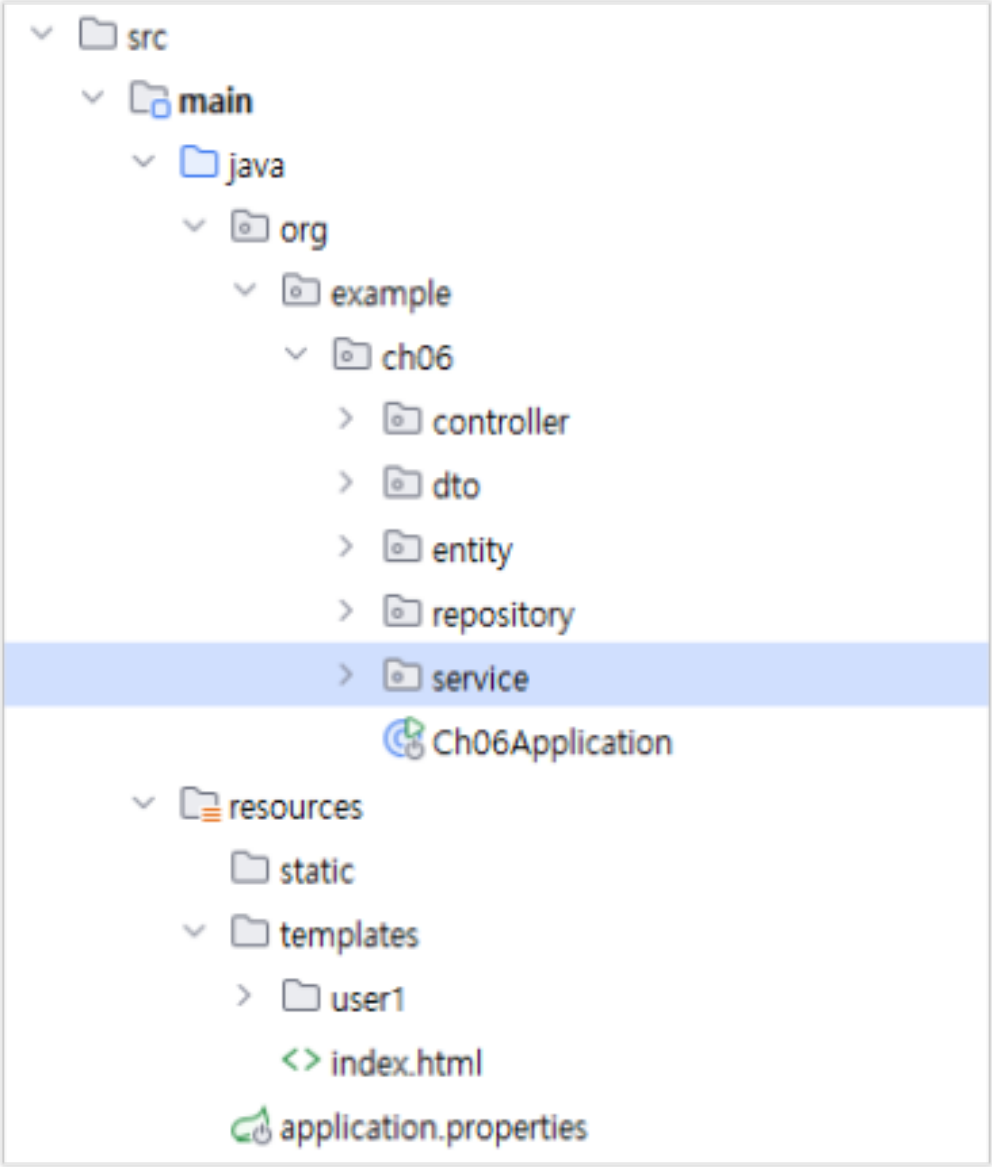
[등록](#)

아이디	이름	생년월일	나이	관리
A101	김유신	010-1234-1111	30	수정 삭제
A102	김춘추	010-1234-2222	23	수정 삭제
A103	장보고	010-1234-3333	32	수정 삭제
A104	강감찬	010-1234-1234	45	수정 삭제
A105	이순신	010-1234-5555	24	수정 삭제

- 개발 목표
- Spring Boot와 JPA를 활용하여 데이터베이스 연동기능 구현
 - JPA를 이용해 데이터의 CRUD 기능 구현
 - 계층형 구조를 적용하여 애플리케이션 개발

- 개발 환경
- JDK 21
 - Spring Boot 3.x
 - Spring Data JPA
 - MySQL Server

02 프로젝트 구조 및 DB 설계



```
CREATE TABLE `user1` (  
  `userid` varchar(255) NOT NULL,  
  `name` varchar(10) NOT NULL,  
  `hp` char(13) NOT NULL,  
  `age` int NOT NULL  
)
```

프로젝트 구조

- controller, dto, entity, repository, service 로 계층형 구조를 적용

User1 테이블 설계

- userid 를 PK 로 설정
- 요구사항에 맞게 NULL 값 허용 X
- 데이터 유형에 맞게 선언

A-Z userid ▼	A-Z name ▼	A-Z hp ▼	123 age ▼
A101	김유신	010-1234-1111	30
A102	김춘추	010-1234-2222	23
A103	장보고	010-1234-3333	32
A104	강감찬	010-1234-1234	45
A105	이순신	010-1234-5555	24

03 CRUD 기능구현 - 등록

```
@PostMapping("/user1/register")
public String register(User1DTO dto) {
    log.info(dto.toString());

    // 서비스 호출
    service.register(dto);

    // 목록으로 리다이렉트
    return "redirect:/user1/list";
}
```

```
private final User1Repository repository;

// 기본 서비스 메서드
public void register(User1DTO dto) { 1 usage
    // DTO를 Entity로 변환
    User1 entity = dto.toEntity();

    // JPA save() API 메서드 호출, Entity가 데이터베이스에 INSERT
    repository.save(entity);
}
```

핵심 흐름

등록 화면 입력 → Controller DTO 수신 → Service Entity 변환 →
repository.save() → DB INSERT

03 CRUD 기능구현 - 목록 조회

```
@GetMapping("/user1/list")
public String list(Model model) {

    // 서비스 호출
    List<User1DTO> dtoList = service.getUserAll();

    // 모델 할조
    model.addAttribute("dtoList", dtoList);

    return "/user1/list";
}
```

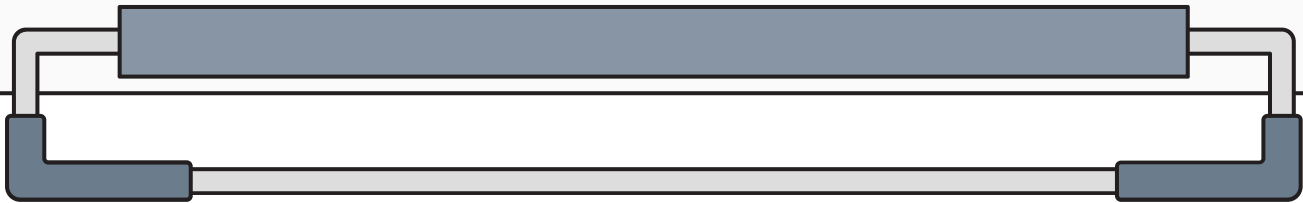
```
<tr th:each="dto:${dtoList}">
  <td th:text="${dto.userid}"></td>
  <td th:text="${dto.name}"></td>
  <td th:text="${dto.hp}"></td>
  <td th:text="${dto.age}"></td>
  <td>
    <a th:href="@{/user1/modify(userid=${dto.userid})}">수정</a>
    <a th:href="@{/user1/remove(userid=${dto.userid})}"
      onclick="return confirm('삭제 하시겠습니까?');">
      삭제
    </a>
  </td>
</tr>
```

```
public List<User1DTO> getUserAll() { 1 usage
    // JPA 조회 메서드 호출, SELECT * FROM User1
    List<User1> entityList = repository.findAll();

    // DTO 리스트로 변환
    //List<User1DTO> dtoList = entityList.stream().map(User1::toDTO).toList();

    return entityList.stream().map(User1::toDTO).toList();
}
```

핵심 흐름
목록 요청 → Controller → Service → repository.findAll() →
DTO 변환 → Model 저장 → Thymeleaf 목록 출력



03 CRUD 기능구현 - 수정

```
@GetMapping("/user1/modify")
public String modify(String userid, Model model) {
    log.info(userid);

    // 서비스 호출
    User1DTO dto = service.getUser(userid);

    // 모델 참조
    model.addAttribute(dto);    // 키 값을 생각하면 소문자로 시작하는 객체 타입이 이름이 됨

    return "/user1/modify";
}

@PostMapping("/user1/modify")
public String modify(User1DTO dto) {
    log.info(dto.toString());

    // 서비스 호출
    service.modify(dto);

    // 목록으로 리다이렉트
    return "redirect:/user1/list?modify=success";
}
```

```
public void modify(User1DTO dto) { 1 usage

    // 엔티티 존재 여부 확인
    boolean isExist = repository.existsById(dto.getUserid());

    // 수정하려는 엔티티가 존재하면 수정
    if(isExist) {
        // DTO를 Entity로 변환
        User1 entity = dto.toEntity();

        // JPA 수정 메서드 호출, save 메서드는 INSERT or UPDATE
        repository.save(entity);
    }
}
```

```
<form th:action="@{/user1/modify}" method="post">
    <table border="1">
        <tr>
            <td>아이디</td>
            <td><input type="text" name="userid" th:value="${user1DTO.userid}" readonly></td>
        </tr>
        <tr>
            <td>이름</td>
            <td><input type="text" name="name" th:value="${user1DTO.name}"></td>
        </tr>
        <tr>
            <td>휴대폰</td>
            <td><input type="text" name="hp" th:value="${user1DTO.hp}"></td>
        </tr>
        <tr>
            <td>나이</td>
            <td><input type="text" name="age" th:value="${user1DTO.age}"></td>
        </tr>
        <tr>
            <td colspan="2" align="right">
                <input type="submit" value="수정하기">
            </td>
        </tr>
    </table>
</form>
```

핵심 흐름
수정 버튼 클릭 → userid 전달 → 기존 데이터 조회 → 수정 화면 출력 → 수정 요청 → save() → DB UPDATE

03 CRUD 기능구현 - 삭제

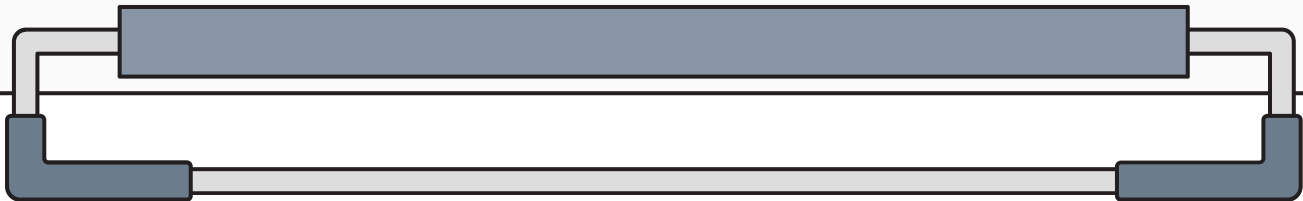
```
<a th:href="@{/user1/remove(userid=${dto.userid})}"  
  onclick="return confirm('삭제 하시겠습니까?');">  
  삭제  
</a>
```

```
@GetMapping("/user1/remove")  
public String remove(String userid) {  
    log.info(userid);  
  
    // 서비스 호출  
    service.remove(userid);  
  
    return "redirect:/user1/list";  
}
```

```
public void remove(String userid) { 1 usage  
  
    // JPA 삭제 메서드 호출, DELETE FROM User1 WHERE userid=?  
    repository.deleteById(userid);  
}
```

핵심 흐름

삭제 버튼 클릭 → 확인 메시지 출력 → 확인 클릭 → Controller →
Service → deleteById() → DB DELETE → 목록 이동



04 실행 결과 - 등록

User1 목록

[메인](#)
[등록](#)

아이디	이름	생년월일	나이	관리
A101	김유신	010-1234-1111	33	수정 삭제
A102	김춘추	010-1234-2222	23	수정 삭제
A103	장보고	010-1234-3333	32	수정 삭제
A104	강감찬	010-1234-1234	45	수정 삭제
A105	이순신	010-1234-5555	24	수정 삭제

User1 등록

[메인](#)
[목록](#)

아이디	test1
이름	테스트1
휴대폰	010-1234-1234
나이	20

등록하기

User1 목록

[메인](#)
[등록](#)

아이디	이름	생년월일	나이	관리
A101	김유신	010-1234-1111	33	수정 삭제
A102	김춘추	010-1234-2222	23	수정 삭제
A103	장보고	010-1234-3333	32	수정 삭제
A104	강감찬	010-1234-1234	45	수정 삭제
A105	이순신	010-1234-5555	24	수정 삭제
test1	테스트1	010-1234-1234	20	수정 삭제

04 실행 결과 - 수정

User1 목록

[메인](#)
[등록](#)

아이디	이름	생년월일	나이	관리
A101	김유신	010-1234-1111	30	수정 삭제
A102	김춘추	010-1234-2222	23	수정 삭제
A103	장보고	010-1234-3333	32	수정 삭제
A104	강감찬	010-1234-1234	45	수정 삭제
A105	이순신	010-1234-5555	24	수정 삭제

User1 수정

[메인](#)
[목록](#)

아이디	A101
이름	김유신
휴대폰	010-1234-1111
나이	33
수정하기	

User1 목록

[메인](#)
[등록](#)

아이디	이름	생년월일	나이	관리
A101	김유신	010-1234-1111	33	수정 삭제
A102	김춘추	010-1234-2222	23	수정 삭제
A103	장보고	010-1234-3333	32	수정 삭제
A104	강감찬	010-1234-1234	45	수정 삭제
A105	이순신	010-1234-5555	24	수정 삭제

04 실행 결과 - 삭제

User1 목록

[메인](#)
[등록](#)

아이디	이름	생년월일	나이	관리
A101	김유신	010-1234-1111	33	수정 삭제
A102	김춘추	010-1234-2222	23	수정 삭제
A103	장보고	010-1234-3333	32	수정 삭제
A104	강감찬	010-1234-1234	45	수정 삭제
A105	이순신	010-1234-5555	24	수정 삭제
test1	테스트1	010-1234-1234	20	수정 삭제

localhost:8080 내용:
삭제 하시겠습니까?

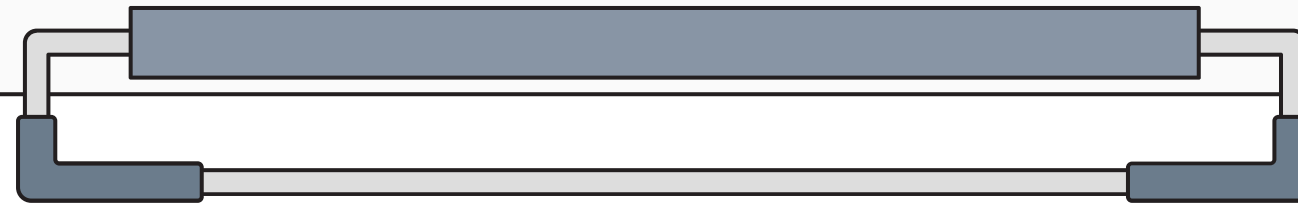
확인

취소

User1 목록

[메인](#)
[등록](#)

아이디	이름	생년월일	나이	관리
A101	김유신	010-1234-1111	33	수정 삭제
A102	김춘추	010-1234-2222	23	수정 삭제
A103	장보고	010-1234-3333	32	수정 삭제
A104	강감찬	010-1234-1234	45	수정 삭제
A105	이순신	010-1234-5555	24	수정 삭제



05 결론

- Spring Boot와 JPA를 이용하여 사용자 정보의 등록, 목록 조회, 수정, 삭제 기능을 구현하였다.
- Controller, Service, Repository 계층을 분리하여 요청 처리, 비즈니스 로직, 데이터베이스 접근 역할을 구분하였다.
- JPA의 save(), findAll(), findById(), deleteById() 메서드를 활용하여 기본 CRUD 기능을 구현하였다.
- DTO와 Entity를 분리하고 변환 메서드를 사용하여 화면 데이터와 데이터베이스 객체를 구분하였다.
- Thymeleaf를 통해 조회한 데이터를 화면에 출력하고, 수정 및 삭제 기능을 사용자 화면과 연결하였다.
- 이번 구현을 통해 Spring MVC와 JPA 기반 CRUD 처리 흐름을 이해할 수 있었다.